CSCE-5683
Digital Image Processing

Assignment 2: Review

Spring 2026

Prof. Khoa Luu

Prepared by Dr. Thanh-Dat Truong

Notes

- The content in the slides is the hints, NOT a solution. Please do not copy and paste directly to your submissions. Please answers the questions of the homework in detail.
- It is totally fine if you do the homework based on the content of the slides.
- You are welcome to do the homework on your own ideas as long as it is correct.
- For programming questions, you are free to use either Python or C++

Notes for OpenCV

- To use SURF and SIFT feature of OpenCV, you should use **Python 3.7** and install the following opencv-python and opencv-contrib-python:
  - **pip install opencv-contrib-python==3.4.2.17 opencv-python==3.4.2.17**

Assignment recap

What the homework is asking you to build and compare

Question 1: Invariant Feature Matching

- Implement or reuse SIFT, SURF, and MINIMA.
- Run them on the image pairs in the provided Q1 folder.
- Report matching accuracy and computational time.
- Goal: compare classical local features vs a newer modality-invariant method.

Question 2: Binary Segmentation

- Implement Otsu thresholding on the Q2 images.
- Compare your masks with ground truth masks.
- Compute PSNR for each image and save result.txt.
- Hint from the assignment: local thresholding may be needed.

Question 3: Multi-class Segmentation

- Reproduce a deep learning-based method: BiMaL.
- Segment each image into n regions.
- Compare predicted regions with the provided GT.
- Focus on correct preprocessing, label format, and evaluation.

Lecture-style roadmap

The deck expands the review into a full implementation guide

Q1 features → Q1 matching → Q2 thresholding → Q2 PSNR → Q3 segmentation → Checklist

Part A — Feature Matching

SIFT and SURF: classic local keypoint pipelines.

MINIMA: stronger for cross-modality or larger appearance gap.

Need: descriptors, matching rule, RANSAC, timing, accuracy table.

Part B — Segmentation

Otsu: Binary threshold from histogram statistics.

BiMaL: Semantic segmentation / Domain Adaptation.

Need: GT comparison, mIoU segmentation metrics, clean outputs.

Question 1 — Invariant Feature Matching

SIFT • SURF • MINIMA

Q1: What the assignment wants

Required deliverables

- For each image pair, produce matched points or a match figure.
- Use SIFT, SURF, and MINIMA on the same test pairs.
- Measure matching accuracy using a consistent rule.
- Measure runtime, preferably per image pair and per method.
- Summarize results in a comparison table and short discussion.

Good report structure

- Method overview: what detector / descriptor each method uses.
- Implementation details: package, threshold, ratio test, RANSAC.
- Quantitative results: matches, inliers, accuracy, time.
- Qualitative results: figures showing correct and incorrect matches.
- Observation: where classical methods fail and MINIMA helps.

Why do we need invariant features?

Important keywords

- Keypoint: a visually distinctive location such as a corner or blob.
- Descriptor: a vector summarizing the neighborhood around a keypoint.
- Invariant: stable under scale, rotation, and modest illumination changes.
- Repeatability: the same physical point is found again in another view.

Why matching is hard

- The same object can appear larger, rotated, darker, or partially occluded.
- Viewpoint change moves pixels, so raw template matching breaks easily.

What a robust method does

- Find salient points, not every pixel.
- Describe local gradients instead of raw intensities.
- Reject ambiguous matches using a ratio test or geometric verification.

General local-feature pipeline

Detect keypoints → Assign scale → Assign orientation → Build descriptor → Match descriptors → Verify geometry

Meaning of each step

- Detection finds candidate points that are distinctive enough to relocate.
- Scale/orientation normalization is what makes the feature more invariant.
- The descriptor turns a patch into a numeric vector for comparison.

Implementation takeaway

- Students rarely code every internal detail from scratch in class projects.
- Using OpenCV implementations is acceptable if your report explains the pipeline and the parameter choices.

SIFT: concepts and keywords

SIFT = Scale-Invariant Feature Transform

What SIFT is doing

- It searches for blob-like extrema in scale space.
- It uses image gradients to create a robust local descriptor.
- Descriptor length is typically 128 dimensions.
- It is strong for scale and rotation changes.

Keywords

- Gaussian pyramid
- Difference of Gaussians (DoG)
- Extrema detection
- Orientation histogram
- Descriptor normalization

Common student mistake

- Only showing the API call without explaining how scale space, orientation, and descriptor construction make the method invariant.

SIFT step 1–2: Scale Space and DoG

These are the concepts behind detecting stable keypoints

Scale space:

- Blur the image with several Gaussian kernels. Small σ keeps fine details; large σ keeps only coarse structures.

DoG (Difference of Gaussians):

- Subtract two blurred images at nearby scales. Peaks and valleys in DoG indicate structures that stand out relative to their neighborhood.

Extremum test:

- A pixel is a candidate keypoint if it is a local maximum or minimum compared with neighbors in x, y, and scale.

Conceptual pseudo code

```
for octave in octaves:
    for sigma in sigmas:
        L[octave, sigma] = gaussian_blur(I, sigma)

for octave in octaves:
    for s in range(num_scales - 1):
        DoG[octave, s] = L[octave, s+1] - L[octave, s]

for each pixel p in DoG:
    if p is max or min in 3x3x3 neighborhood:
        mark as candidate keypoint
```

SIFT step 3–5: orientation and descriptor

Why SIFT keeps working after rotation

Orientation assignment

- Compute gradient magnitude and direction near the keypoint.
- Build a histogram of gradient directions.
- Use the dominant direction as the keypoint orientation.
- Now the patch can be rotated into a canonical frame.

Descriptor construction

- Take a window around the keypoint.
- Split it into a 4×4 grid of cells.
- For each cell, build an 8-bin gradient histogram.
- 4×4×8 = 128 values.

Normalization

- Normalize the descriptor vector so that moderate brightness changes have less effect. This is one reason SIFT is more robust than raw pixels.

Concrete SIFT pseudo code

Pseudo code

```
Input: grayscale images I1, I2

# 1) detect and describe
kp1, des1 = SIFT(I1)
kp2, des2 = SIFT(I2)

function SIFT(I):
    build Gaussian pyramid L(x, y, sigma)
    compute DoG pyramid D = L(s+1) - L(s)
    find local extrema in D across (x, y, scale)
    remove low-contrast and unstable points
    for each surviving keypoint:
        compute dominant orientation from local gradients
        rotate local patch to canonical orientation
        divide patch into 4x4 cells
        for each cell:
            accumulate 8-bin gradient histogram
        concatenate 16 histograms -> 128D descriptor
        normalize descriptor
    return keypoints, descriptors

Output: keypoints and 128D descriptors for each image
```

SIFT starter code in OpenCV

A practical baseline students can run first

Detection + descriptor extraction

```
import cv2
import numpy as np

img1 = cv2.imread(path1, cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread(path2, cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
```

Matching with Lowe ratio test

```
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
pairs = bf.knnMatch(des1, des2, k=2)

good = []
for m, n in pairs:
    if m.distance < 0.75 * n.distance:
        good.append(m)

vis = cv2.drawMatches(img1, kp1, img2, kp2,
                      good, None,

flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

Hint: if you also run RANSAC after this step, report both raw matches and inlier matches.

SURF: concepts and keywords

SURF = Speeded-Up Robust Features

Big idea

- SURF tries to keep SIFT-like robustness while running faster.
- It approximates second-order structure using the Hessian matrix.
- Integral images make box-filter responses very fast to compute.

Keywords

- Integral image
- Box filter approximation
- Hessian determinant
- Orientation from Haar-wavelet responses
- 64D descriptor (common version)

Comparison intuition

- SIFT emphasizes rich gradient histograms
- SURF emphasizes speed through approximations.

How SURF detects and describes points

Detector:

- SURF evaluates the determinant of an approximated Hessian matrix. Strong blob-like structures create high responses.

Why integral images help:

- Box filters over rectangular regions can be computed very quickly, so different scales become cheaper to evaluate.

Descriptor:

- SURF uses Haar-wavelet responses in x and y inside subregions. These responses summarize local intensity changes with lower cost than SIFT's full histogram descriptor.

Conceptual pseudo code

```
for each scale:
    compute box-filter approximation to Hessian
    score = det(Hessian_approx)
    keep local maxima as candidate keypoints

for each keypoint:
    estimate orientation from Haar responses
    divide neighborhood into subregions
    summarize dx, dy, |dx|, |dy|
    concatenate to descriptor
```

Concrete SURF pseudo code

Pseudo code

```
Input: grayscale images I1, I2

# 1) integral images
II1 = integral_image(I1)
II2 = integral_image(I2)

# 2) detect + describe
kp1, des1 = SURF(II1)
kp2, des2 = SURF(II2)

function SURF(II):
    for each scale:
        compute box-filter approximation of Hessian determinant
        detect local maxima in scale space
    for each keypoint:
        estimate dominant orientation using Haar-wavelet responses
        divide local region into sub-blocks
        for each block, compute sums of dx, dy, |dx|, |dy|
        concatenate features into descriptor
    return keypoints, descriptors

# 3) match descriptors using nearest-neighbor + ratio test
```

SURF starter code in OpenCV

Detection + descriptor extraction

```
import cv2

img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

surf = cv2.xfeatures2d.SURF_create(
    hessianThreshold=400,
    nOctaves=4,
    nOctaveLayers=3,
    extended=False,   # 64D descriptor
    upright=False
)

kp, des = surf.detectAndCompute(img, None)
```

Practical note

- SURF is patented in some contexts, so it may be unavailable in default OpenCV wheels.
- If SURF is missing, use opencv-contrib-python or an alternative open-source implementation.
- Use the same matching rule you used for SIFT so the comparison stays fair.

MINIMA: concepts and keywords

MINIMA is a recent method for modality-invariant image matching

What problem MINIMA targets

- Classical descriptors work best when both images come from similar appearance statistics.
- Cross-modal pairs (e.g., RGB–IR, optical–SAR, RGB–depth) create a modality gap.
- MINIMA learns features that are more invariant across such gaps.

Keywords

- Cross-modality
- Feature embedding
- Correspondence
- Generalization
- Data scaling / data engine

Notes

- You are not expected to re-invent the model internals. Reproducing the provided code pipeline and evaluating it correctly is the practical goal.

How to reproduce MINIMA for homework

A clean, honest strategy for a modern paper reproduction

Suggested workflow

- Clone the official repository or use the released checkpoints.
- Prepare the same pair format expected by the inference script.
- Run the model on the assignment image pairs.
- Extract predicted correspondences / matches / score maps.
- Measure runtime using the same environment for all methods when possible.

Pseudo workflow

```
# conceptual reproduction plan
git clone <MINIMA_repo>
install dependencies
download checkpoint

for each image pair:
    preprocess to expected size / format
    run model inference
    parse predicted matches
    save visualization
    record runtime
```

Good report wording: “We reproduced the released inference pipeline and evaluated it on the assignment pairs.”

Concrete experiment pseudo code for SIFT / SURF / MINIMA

Comparison loop

```
methods = ["SIFT", "SURF", "MINIMA"]
results = []

for pair in image_pairs:
    I1, I2 = load_pair(pair)

    for method in methods:
        t0 = now()

        if method == "SIFT":
            kp1, des1 = run_sift(I1)
            kp2, des2 = run_sift(I2)
            matches = match_with_ratio_test(des1, des2)

        elif method == "SURF":
            kp1, des1 = run_surf(I1)
            kp2, des2 = run_surf(I2)
            matches = match_with_ratio_test(des1, des2)

        elif method == "MINIMA":
            matches = run_minima_inference(I1, I2)

        inliers = geometric_verification(matches)
        acc = compute_accuracy(inliers, ground_truth_if_available)
        dt = now() - t0

        results.append([pair.name, method, len(matches), len(inliers), acc, dt])
```

Descriptor matching: nearest neighbor + ratio test

This is usually the most important post-processing detail for SIFT/SURF

Nearest neighbor

- For each descriptor in image 1, find the closest descriptor in image 2.
- Distance is often Euclidean (L2) for SIFT and SURF.

Why ratio test works

- If the best and second-best matches are almost equally close, the point is ambiguous.
- Keep a match only if best_distance < r × second_best_distance, often r = 0.75.

Concrete pseudo code

```
pairs = bf.knnMatch(des1, des2, k=2)
good = []
for m, n in pairs:
    if m.distance < 0.75 *
n.distance:
        good.append(m)
```

Geometric verification with RANSAC

Why it matters:

- Descriptors are local. Some incorrect pairs can still have similar feature vectors. RANSAC checks whether the matches agree with one global geometric transform.

Typical use:

- Estimate a homography or fundamental matrix from matched points. Matches consistent with the estimated model are called inliers.

In a report:

- You can show both raw matches and RANSAC-filtered inliers.

Starter code

```
pts1 = np.float32([kp1[m.queryIdx].pt for m in good])
pts2 = np.float32([kp2[m.trainIdx].pt for m in good])

H, mask = cv2.findHomography(
    pts1, pts2, cv2.RANSAC, 5.0
)

inliers = [m for m, keep in zip(good, mask.ravel()) if
keep]
```

Question 2 — Binary Segmentation

Otsu thresholding

Q2: what the assignment wants

Required deliverables

- Run Otsu thresholding on each input image.
- Compare the segmentation result with its ground truth image.
- Compute PSNR for every image.
- Write results to result.txt in the required format.

What the note in the assignment means

- A single global threshold may fail if the illumination changes across the image.
- In that case, use a local strategy: divide the image into windows or use adaptive thresholding.
- Explain clearly whether you used global Otsu, local Otsu, or adaptive thresholding as an improvement.

Binary segmentation: concepts and keywords

Thresholding

- Convert grayscale intensities into two classes: foreground and background.
- Rule: pixel > T -> foreground, else background.

Histogram

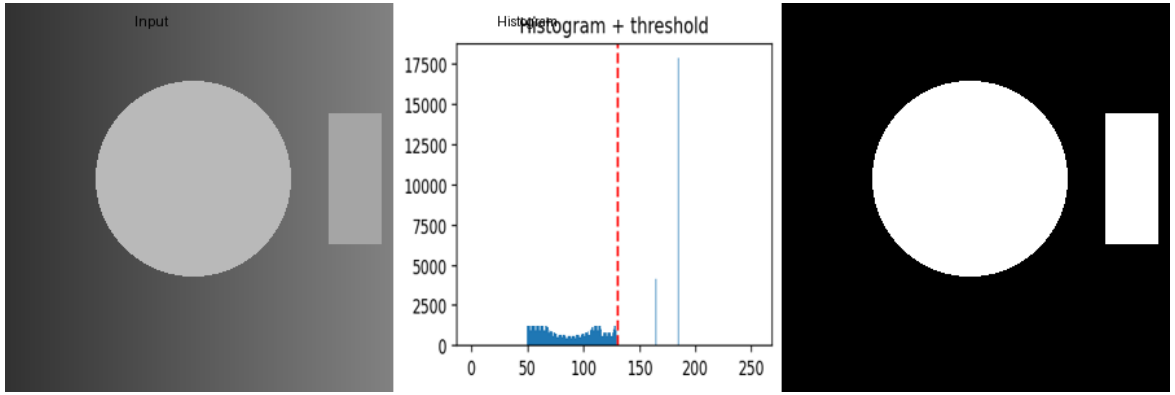
- A histogram counts how many pixels have each intensity value.
- If the image is clean and bimodal, a good threshold often lies between two peaks.

Ground truth (GT)

- The GT mask is the reference segmentation.
- Your output must be aligned in size and pixel convention before computing PSNR.

Otsu method: intuition before equations

Otsu automatically chooses the threshold from histogram statistics



How to explain Otsu

- Try every possible threshold T from 0 to 255.
- Split the histogram into class 0 and class 1 using T.
- Compute how separate the two classes are.
- Choose the threshold that maximizes between-class variance.
- Equivalent view: choose T that minimizes within-class confusion.

Concrete Otsu pseudo code

Pseudo code

```
Input: grayscale image I

hist = histogram(I, bins=256)
best_T = 0
best_score = -inf

for T in range(256):
    class0 = hist[0:T+1]
    class1 = hist[T+1:256]

    w0 = sum(class0) / total_pixels
    w1 = sum(class1) / total_pixels
    if w0 == 0 or w1 == 0:
        continue

    mu0 = weighted_mean(class0)
    mu1 = weighted_mean(class1)

    sigma_b2 = w0 * w1 * (mu0 - mu1)**2

    if sigma_b2 > best_score:
        best_score = sigma_b2
        best_T = T

mask = I > best_T
```

Keyword explanations

- w0, w1: fraction of pixels in each class
- mu0, mu1: class means
- sigma_b^2: between-class variance
- Large separation means a better threshold

When global Otsu fails: local strategy

Why global threshold fails

Shadows, vignetting, or non-uniform illumination make one threshold too small for some regions and too large for others.

Local Otsu idea

```
for each window in image:
    compute Otsu threshold T_local
    threshold only that window
merge all window results
optionally smooth boundaries
```

Practical OpenCV

```
seg = cv2.adaptiveThreshold(
    img, 255,

cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=31,
    C=5
)
```

PSNR: what it measures and how to compute it

PSNR is derived from the pixel-wise error between your mask and the GT mask

Step 1: mean squared error

MSE = average of (prediction - ground_truth)^2 over all pixels.

If the masks are identical, MSE = 0.

Step 2: convert to PSNR

PSNR = 10 log10(MAX^2 / MSE)

For 8-bit images, MAX = 255.

Higher PSNR means lower error.

Starter code

```
pred = pred.astype(np.float32)
gt   = gt.astype(np.float32)

mse = np.mean((pred - gt) ** 2)
if mse == 0:
    psnr = float("inf")
else:
    psnr = 10 * np.log10((255 ** 2)
/ mse)
```

Q2 starter script skeleton + result.txt format

Batch-processing skeleton

```
results = []

for img_path, gt_path in paired_files:
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gt  = cv2.imread(gt_path,  cv2.IMREAD_GRAYSCALE)

    # choose one:
    # seg = global_otsu(img)
    # seg = local_otsu(img)
    seg = run_threshold_method(img)

    psnr = compute_psnr(seg, gt)
    results.append((os.path.basename(img_path), psnr))

with open("result.txt", "w") as f:
    for name, psnr in results:
        f.write(f"{name} {psnr:.4f}\n")
```


Important: make sure pred and GT use the same convention (0/255 or 0/1) before PSNR.

Question 3 — Multi-Class Segmentation

BiMaL

Q3: what the assignment wants

This question is about multi-class / multi-region segmentation

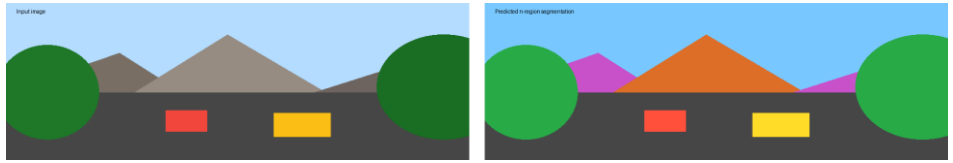
Task interpretation

- Each image must be segmented into n regions, not just foreground/background.
- The output is a label map: each pixel belongs to one of n classes.
- You must reproduce a deep-learning-based approach using BiMaL.

Practical project goal

- Use an available implementation or pretrained pipeline if possible.
- Understand the data format and the inference steps.
- Focus on producing comparable segmentation maps and evaluating them against GT.

Semantic segmentation: key concepts



Key terms

Class label

Logits

Softmax

Argmax

mIoU

Pixel accuracy

What the output is

For each pixel, the model predicts a score for each class. The final region ID is usually argmax over class scores.

BiMaL intuition

Problem setting

- Semantic segmentation models trained on one domain often fail on another domain.
- BiMaL is designed for domain adaptation in semantic scene segmentation.

High-level intuition

- The method encourages predictions that better capture the global structure of the segmentation map, not only independent per-pixel confidence.

Notes

- For homework, it is acceptable to focus on reproducing the released training / inference pipeline and reporting results carefully.

Before running a segmentation model: preprocessing and labels

Input preprocessing

- Resize to the model's expected resolution.
- Convert BGR↔RGB correctly.
- Normalize using the mean/std expected by the training code.
- Keep batch and channel dimensions correct: [B, C, H, W].

Ground-truth labels

- GT is usually an integer label map, not a color image.
- Do not normalize class IDs like image intensities.
- Check that label values are from 0 to n-1.

Starter preprocessing code

```
img = cv2.imread(path)[:, :, ::-1]  # BGR->RGB
img = cv2.resize(img, (W, H))
img = img.astype(np.float32) / 255.0
img = (img - mean) / std
img = np.transpose(img, (2, 0, 1))
img = torch.from_numpy(img).unsqueeze(0)
```

PyTorch inference starter code for segmentation

Inference skeleton

```
import torch

model.eval()
with torch.no_grad():
    x = preprocess(image)              # [1, C, H, W]
    logits = model(x)                  # [1, n_classes, H, W]
    if isinstance(logits, dict):       # torchvision-style output
        logits = logits["out"]
    pred = torch.argmax(logits, dim=1) # [1, H, W]
    pred = pred.squeeze(0).cpu().numpy()

# optional colorization for visualization
color_mask = colorize(pred)
```

Q3 Pseudo code for evaluation

Evaluation pseudo code

```
for image, gt in dataset:
    x = preprocess(image)
    logits = model(x)
    pred = argmax(logits, axis=class)

    # evaluation examples
    pixel_acc = mean(pred == gt)
    iou = intersection_over_union(pred, gt)
    save_visualization(image, pred, gt)

summarize mean metrics over all images
```

Final checklist

Final checklist

Q1: compare SIFT, SURF, MINIMA with runtime and accuracy.

Q2: generate masks, compute PSNR, save result.txt.

Q3: produce n-region maps and compare with GT.

Every section should include both visuals and metrics, parameters, libraries clearly.